

Predictive Parsing

Like recursive-descent but parser can "Predict" which production to use

- By looking at the next few tokens.
(lookahead restricted grammars)
- No backtracking

Predictive parsers accept $LL(k)$ grammars

left-to-right scan

left-most $L > k$ tokens
Lookahead ($k=1$)

In recursive descent,

- At each step, many choices of production to use.
- Backtracking used to undo bad choices.

in $LL(1)$,

- At each step, only one choice of production

Based on LL(1) [only 1 token]

- Recall the grammar

$$\begin{aligned} E &\rightarrow \underline{T} + E \mid \underline{T} \\ T &\rightarrow \underline{\text{int}} \mid \underline{\text{int}} * T \mid (E) \end{aligned}$$

- Hard to predict because
 - For T two productions start with int
 - For E it is not clear how to predict
- We need to left-factor the grammar

- Left-factored grammar

$$\begin{aligned} E &\rightarrow T X \\ T &\rightarrow (E) \mid \text{int } Y \\ X &\rightarrow + E \mid \epsilon \\ Y &\rightarrow * T \mid \epsilon \end{aligned}$$

- The LL(1) parsing table:

	int	*	+	(	)	\$
E	$T X$			$T X$		
X			$+ E$		ϵ	ϵ
T	$\text{int } Y$			(E)		
Y		$* T$	ϵ		ϵ	ϵ

leftmost non-terminal

next input token

rhs of production to use

right hand side

- Method similar to recursive descent, except
 - For the leftmost non-terminal S
 - We look at the next input token a
 - And choose the production shown at [S,a]
- A stack records frontier of parse tree
 - Non-terminals that have yet to be expanded
 - Terminals that have yet to be matched against the input
 - Top of stack = leftmost pending terminal or non-terminal
- Reject on reaching error state
- Accept on end of input & empty stack

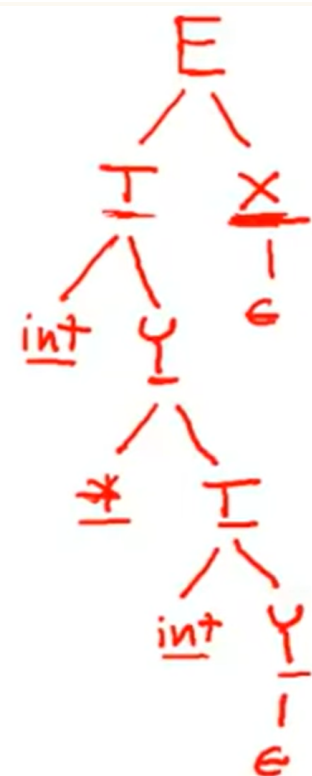
initialize stack = <S \$> and next end of input
 repeat

case stack of

<X, rest> : if T[X, *next] = Y₁...Y_n
 then stack $\leftarrow$ <Y₁... Y_n rest>;
 else error();
<t, rest> : if t == *next ++
 then stack $\leftarrow$ <rest>;
 else error();

until stack == <>

Stack	Input	Action
<u>E</u> \$	<u>int</u> * int \$	<u>T X</u>
<u>T</u> X \$	<u>int</u> * int \$	<u>int Y</u>
<u>int</u> Y X \$	<u>int</u> * int \$	terminal
<u>Y</u> X \$	<u>*</u> int \$	<u>* T</u>
<u>*</u> X \$	<u>*</u> int \$	terminal
<u>X</u> \$	<u>int</u> \$	<u>int Y</u>
<u>int</u> Y X \$	<u>int</u> \$	terminal
<u>Y</u> X \$	\$	<u>ϵ</u>
<u>X</u> \$	\$	<u>ϵ</u>
<u>\$</u>	\$	<u>ACCEPT</u>



First sets

- Consider non-terminal A , production $A \rightarrow \alpha$, & token t

$T[A, t] = \alpha$ in two cases:

If $\alpha \xrightarrow{*} t \beta$

- α can derive a t in the first position
- We say that $t \in \text{First}(\alpha)$

→ first set

$$\left[\begin{array}{l} \alpha \not\xrightarrow{*} t \beta \\ t \notin \text{First}(\alpha) \end{array} \right]$$

If $A \rightarrow \alpha$ and $\alpha \xrightarrow{*} \epsilon$ and $S \xrightarrow{*} \beta A t \delta$

- Useful if stack has A , input is t , and A cannot derive t
- In this case only option is to get rid of A (by deriving ϵ)
 - Can work only if t can follow A in at least one derivation
- We say $t \in \text{Follow}(A)$

→ follow set

Definition

$$\text{First}(X) = \{ t \mid X \xrightarrow{*} t \alpha \} \cup \{ \epsilon \mid X \xrightarrow{*} \epsilon \}$$

Algorithm sketch:

1. $\text{First}(t) = \{ t \}$ t is a terminal

2. $\epsilon \in \text{First}(X)$

- if $X \rightarrow \epsilon$
- if $X \rightarrow A_1 \dots A_n$ and $\epsilon \in \text{First}(A_i)$ for $1 \leq i \leq n$

3. $\text{First}(\alpha) \subseteq \text{First}(X)$ if $X \rightarrow A_1 \dots A_n \alpha$

– and $\epsilon \in \text{First}(A_i)$ for $1 \leq i \leq n$

- Recall the grammar

$$E \rightarrow T X$$

$$T \rightarrow (E) \mid \text{int } Y$$

$$X \rightarrow + E \mid \epsilon$$

$$Y \rightarrow * T \mid \epsilon$$

$$X \rightarrow A_1 \dots A_n \xrightarrow{*} \alpha$$

$$X \xrightarrow{*} \epsilon$$

$$\text{First}(+) = \{ + \}$$

$$\text{First}(*) = \{ * \}$$

$$\vdots \quad \{ (\}$$

$$\vdots \quad \{) \}$$

$$\{ \text{int} \}$$

$$\text{First}(E) = \text{First}(T)$$

$$\text{First}(T) = \{ (, \text{int} \}$$

$$\text{First}(X) = \{ +, \epsilon \}$$

$$\text{First}(Y) = \{ *, \epsilon \}$$

Follow Sets

- Definition:

$$\text{Follow}(X) = \{ \underline{t} \mid S \rightarrow^* \beta \underline{X} \underline{t} \delta \}$$

$$\underline{S} \rightarrow \underline{X} \underline{t} \rightarrow \underline{ABt} \rightarrow \underline{A} \underline{t}$$

$$X \rightarrow AB \rightarrow A \underline{t} B$$

- Intuition

– If $\underline{X} \rightarrow \underline{A} \underline{B}$ then $\underline{\text{First}}(B) \subseteq \underline{\text{Follow}}(A)$ and

$$\underline{\text{Follow}}(X) \subseteq \underline{\text{Follow}}(B)$$

• if $\underline{B} \rightarrow^* \varepsilon$ then $\underline{\text{Follow}}(X) \subseteq \underline{\text{Follow}}(A)$

– If $\underline{S}$ is the start symbol then $\underline{\$} \in \underline{\text{Follow}}(\underline{S})$

Algorithm sketch:

1. $\underline{\$} \in \underline{\text{Follow}}(\underline{S})$

2. $\underline{\text{First}}(\beta) - \{\varepsilon\} \subseteq \underline{\text{Follow}}(X)$

– For each production $A \rightarrow \alpha \underline{X} \beta$

3. $\underline{\text{Follow}}(A) \subseteq \underline{\text{Follow}}(X)$

– For each production $A \rightarrow \alpha \underline{X} \underline{\beta}$ where

$$\varepsilon \in \underline{\text{First}}(\underline{\beta})$$

Recall the grammar:

$$E \rightarrow TX \quad X \rightarrow +E \mid \epsilon$$

$$T \rightarrow (E) \mid \text{int } Y \quad Y \rightarrow *T \mid \epsilon$$

$$\text{Follow}(E) = \{ \$,) \}$$

$$\text{Follow}(E) \supseteq \text{Follow}(X)$$

$$\text{Follow}(X) = \{ \$,) \}$$

$$\text{Follow}(X) \supseteq \text{Follow}(E)$$

$$\text{Follow}(T) = \{ +, \$,) \}$$

$$\text{Follow}(T) \supseteq \text{Follow}(Y)$$

$$\text{Follow}(T) \supseteq \text{Follow}(E)$$

$$\text{Follow}(Y) = \{ +, \$,) \}$$

$$\text{Follow}(Y) \supseteq \text{Follow}(T)$$

$$\text{Follow}('(') = \{ (, \text{int} \}$$

$$\text{Follow}(')') = \{ +, \$,) \}$$

$$\text{Follow}('+') = \{ (, \text{int} \}$$

$$\text{Follow}('*') = \{ (, \text{int} \}$$

$$\text{Follow}(\text{int}) = \{ *, +, \$,) \}$$

$$\rightarrow \text{Follow}(E) = \text{Follow}(X)$$

$$\rightarrow \text{Follow}(T) = \text{Follow}(Y)$$

LL(1) Parsing Table

Construct a parsing table T for CFG G

For each production $A \rightarrow \alpha$ in G do:

① For each terminal $t \in \text{First}(\alpha)$ do

$$T[A, t] = \alpha$$

② If $\epsilon \in \text{First}(\alpha)$, for each $t \in \text{Follow}(A)$
do $T[A, t] = \alpha$

③ If $\epsilon \in \text{First}(\alpha)$, for each $\$ \in \text{Follow}(A)$
do $T[A, \$] = \alpha$

If any entry is multiply defined then G is not LL(1)

[not left factored
left recursive
ambiguous

other grammars are not LL(1)

Most Programming language CFGs are not LL(1).

Bottom-Up Parsing

Bottom-up parsing is more general than (deterministic) top-down parsing.

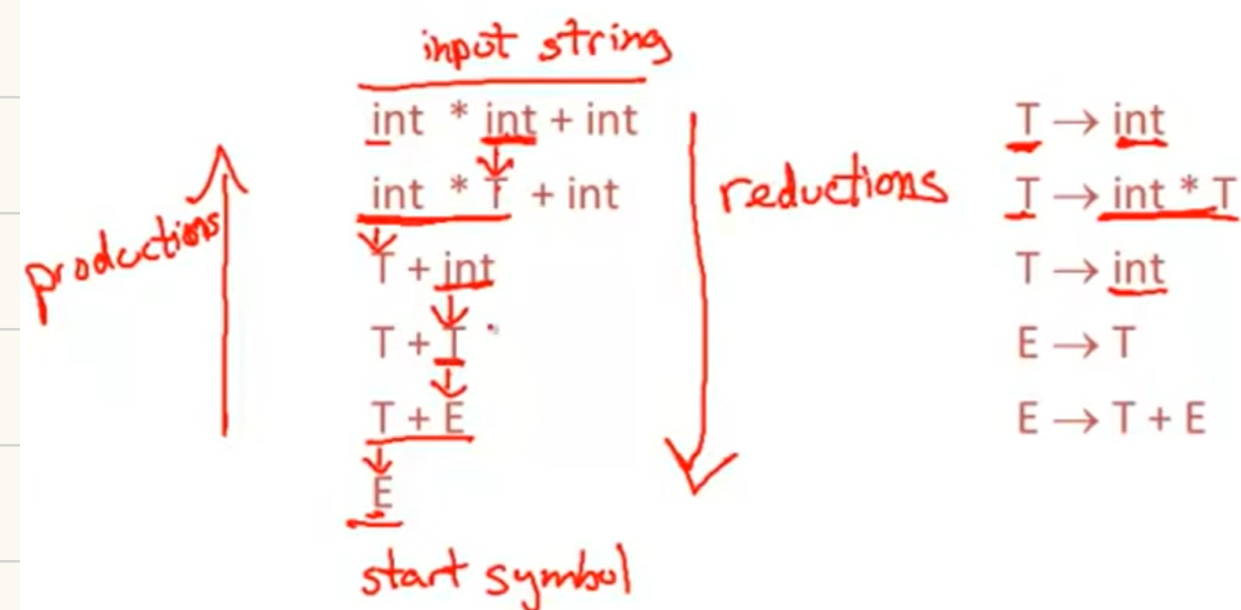
And just as efficient

Builds on ideas in top-down parsing

Bottom-up parsing is the preferred method

Bottom-up parsers don't need left-factored grammars.

Bottom-up parsing reduces a string to the start symbol by inverting productions



Important Fact #1 about bottom-up
Parsing:

A bottom-up parser traces a rightmost
derivation in reverse

Shift-Reduce Parsing

Important Fact #1 has an interesting consequence:

- Let $\underline{\alpha}\underline{\beta}\underline{\omega}$ be a step of a bottom-up parse
- Assume the next reduction is by $\underline{X} \xrightarrow{\leftarrow} \underline{\beta}$
- Then $\underline{\omega}$ is a string of terminals

Why? Because $\alpha\underline{X}\underline{\omega} \xrightarrow{\leftarrow} \alpha\underline{\beta}\omega$ is a step in a right-most derivation

- Idea: Split string into two substrings
 - Right substring is as yet unexamined by parsing
 - Left substring has terminals and non-terminals
 - The dividing point is marked by a $|$

$\alpha X \omega$ 

- *Shift*: Move | one place to the right
 - Shifts a terminal to the left string

$$ABC \mid xyz \Rightarrow ABC \underline{x} \mid \underline{y}z$$

- Apply an inverse production at the right end of the left string
 - If $A \rightarrow \underline{xy}$ is a production, then

$$Cb \underline{xy} \mid ijk \Rightarrow Cb \underline{A} \mid ijk$$

- Left string can be implemented by a stack
 - Top of the stack is the |
- Shift pushes a terminal on the stack
- Reduce
 - pops symbols off of the stack (production rhs)
 - pushes a non-terminal on the stack (production lhs)
- In a given state, more than one action (shift or reduce) may lead to a valid parse
- If it is legal to shift or reduce, there is a shift-reduce conflict
- If it is legal to reduce by two different productions, there is a reduce-reduce conflict Back